

Docker cơ bản cho người mới



Dockerfile



Image



Cài Docker trên Windows

> DOCS

<https://docs.docker.com/desktop/setup/install/windows-install/>

EXPECTED RESULT Download Docker Desktop Installer.exe

> VERIFY

```
docker --version
docker run hello-world
```

EXPECTED RESULT Docker CLI chạy được và pull được image đầu tiên



OFFICIAL DOCS

<https://docs.docker.com/desktop/setup/install/windows-install/>



LƯU Ý

Cần mở Docker Desktop trước khi sử dụng để start engine.

Cài Docker trên Linux

Với Linux, hãy mở tài liệu Docker, chọn đúng bản Linux đang dùng như Ubuntu, Debian hoặc Fedora, rồi cài theo từng bước.

>_ DOCS

<https://docs.docker.com/engine/install/>

EXPECTED RESULT Chọn đúng bản Linux đang dùng

>_ VERIFY

```
docker --version
docker compose version
docker run hello-world
```

EXPECTED RESULT Engine, Compose plugin và môi trường chạy container hoạt động



OFFICIAL DOCS

<https://docs.docker.com/engine/install/>



Vấn đề hiện tại

Một app có thể chạy tốt trên máy A nhưng lỗi trên máy B vì khác version môi trường chạy, thư viện, biến môi trường hoặc cách cài đặt.



Host machine

 **Code**

 **Image**

 **Cùng môi trường**



TRƯỚC DOCKER

Cài Python, Node, database, package và config trực tiếp lên máy hoặc server.



SAU DOCKER

Mô tả môi trường bằng file, build thành image, rồi chạy image đó ở nhiều nơi.

Container là gì

Container là một gói chạy app, bên trong có app và những thứ cần thiết để app chạy ổn định ở nhiều môi trường, nhưng không mang cả hệ điều hành đầy đủ.



Host machine

 **App**

 **Dependency**

 **Cấu hình**



TÓM TẮT

Container là cách đóng gói app cùng dependency và cấu hình chạy, để app dễ mang sang máy khác và ít lỗi do thiếu môi trường, container nhẹ hơn vì không mang cả hệ điều hành đầy đủ.

Dockerfile, Image, Container

Ba khái niệm này thường đi theo một dòng: viết Dockerfile, build hoặc tải Image, rồi chạy Image thành Container.

01



Dockerfile

02



Image

03



Container



DOCKERFILE

File hướng dẫn Docker cách tạo image, gồm base image, copy code, cài



IMAGE

Gói bất biến dùng để tạo container, có thể build từ Dockerfile hoặc tải từ



CONTAINER

Bản đang chạy từ image, có trạng thái, log, port, env và vòng đời riêng

Docker CLI

>_ **VERSION**

```
docker --version
```

CHỨC NĂNG

Kiểm tra Docker CLI đã được cài và xem version đang dùng.

>_ **LIST IMAGES**

```
docker images
```

CHỨC NĂNG

Liệt kê image local, gồm repository, tag, image id và dung lượng.

>_ **RUN BACKGROUND**

```
docker run -d --name api -p 3000:3000 demo-api
```

CHỨC NĂNG

Chạy container ở chế độ nền để tiếp tục dùng terminal.

>_ **COMPOSE VERSION**

```
docker compose version
```

CHỨC NĂNG

Kiểm tra Docker Compose plugin đã sẵn sàng để chạy nhiều service.

>_ **BUILD IMAGE**

```
docker build -t demo-api .
```

CHỨC NĂNG

Build image mới từ Dockerfile trong thư mục hiện tại.

>_ **STOP/REMOVE**

```
docker stop web
docker rm web
```

CHỨC NĂNG

Dừng container đang chạy rồi xóa container khi không cần nữa.

>_ **PULL IMAGE**

```
docker pull nginx:alpine
```

CHỨC NĂNG

Tải image có sẵn từ registry về máy local.

>_ **RUN CONTAINER**

```
docker run --name web -p 8080:80 nginx:alpine
```

CHỨC NĂNG

Tạo container từ image và map port container ra máy host, lệnh này chạy foreground.

>_ **REMOVE IMAGE**

```
docker rmi demo-api
```

CHỨC NĂNG

Xóa image local sau khi container dùng image đó đã được dừng và xóa.

Docker debug

>_

RUNNING CONTAINERS

```
docker ps
```

CHỨC NĂNG
Xem các container đang chạy và port đang được publish.

>_

FOLLOW LOGS

```
docker logs -f <container>
```

CHỨC NĂNG
Theo dõi log realtime khi đang gửi request hoặc debug lỗi.

>_

STATS

```
docker stats
```

CHỨC NĂNG
Theo dõi CPU, memory và network của container đang chạy.

>_

PRUNE

```
docker system prune
```

CHỨC NĂNG
Dọn tài nguyên Docker không còn dùng, cần đọc prompt trước khi xác nhận vì có thể xóa cache, network hoặc image không dùng.

>_

ALL CONTAINERS

```
docker ps -a
```

CHỨC NĂNG
Xem cả container đã dừng, exited hoặc lỗi khi start.

>_

SHELL

```
docker exec -it <container> sh
```

CHỨC NĂNG
Mở shell trong container để kiểm tra file, env hoặc network, nếu image có shell.

>_

VOLUMES

```
docker volume ls
```

CHỨC NĂNG
Liệt kê volume Docker đang quản lý trên máy local.

>_

LOGS

```
docker logs <container>
```

CHỨC NĂNG
Đọc stdout và stderr của app bên trong container.

>_

INSPECT

```
docker inspect <container>
```

CHỨC NĂNG
Xem chi tiết cấu hình container như port, env, mount và network.

>_

NETWORKS

```
docker network ls
```

CHỨC NĂNG
Liệt kê network Docker, hữu ích khi kiểm tra kết nối giữa container.

Docker Compose

>_ VALIDATE CONFIG

```
docker compose config
```

CHỨC NĂNG

Kiểm tra compose.yml sau khi merge biến môi trường và default.

>_ REBUILD START

```
docker compose up --build -d
```

CHỨC NĂNG

Build lại image rồi start các service Compose, hữu ích sau khi sửa code.

>_ EXEC

```
docker compose exec api sh
```

CHỨC NĂNG

Vào shell của service api đang chạy, nếu image của service có shell.

>_ DOWN

```
docker compose down
```

CHỨC NĂNG

Dừng và xóa container, network của Compose project.

>_ BUILD

```
docker compose build
```

CHỨC NĂNG

Build image cho các service có khai báo build context.

>_ STATUS

```
docker compose ps
```

CHỨC NĂNG

Xem trạng thái container thuộc Compose project hiện tại.

>_ RESTART

```
docker compose restart api
```

CHỨC NĂNG

Restart một service mà không cần dừng toàn bộ Compose project.

>_ DOWN VOLUMES

```
docker compose down -v
```

CHỨC NĂNG

Dừng Compose project và xóa cả volume, chỉ dùng khi muốn reset dữ liệu local.

>_ START SERVICES

```
docker compose up -d
```

CHỨC NĂNG

Tạo network, volume, container và chạy các service Compose ở chế độ nền.

>_ LOGS

```
docker compose logs api --tail=40
```

CHỨC NĂNG

Xem log service api với số dòng gần nhất để debug nhanh.

>_ STOP

```
docker compose stop
```

CHỨC NĂNG

Dừng container của Compose project nhưng giữ lại container và network.

Code → Build → Run → Access

Đây là vòng lặp thực hành quan trọng nhất. Mỗi lần sửa app hoặc Dockerfile, mình build image mới rồi chạy container để kiểm tra.

01



Code

02



Build

03



Run

04



Dockerfile tối thiểu cần gì?

Dockerfile nên dễ đọc như một checklist build app. Với beginner, chỉ cần nắm vài instruction xuất hiện nhiều nhất.

> DOCKERFILE

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["npm", "start"]
```

EXPECTED RESULT Image có môi trường chạy Node, dependency và source code



FROM

Chọn base image, ví dụ node:20-alpine hoặc python:3.12-slim.



COPY

Đưa source code hoặc file dependency từ máy host vào image.



RUN

Khi container không chạy

>_ COMMAND

```
docker ps -a
```

EXPECTED RESULT Xem status Up hoặc Exited

>_ COMMAND

```
docker logs <container>
```

EXPECTED RESULT Xem lỗi app hoặc startup log

>_ COMMAND

```
docker exec -it <container> sh
```

EXPECTED RESULT Vào shell nếu image có shell



DOCKER PS -A

Xem container đang chạy hoặc đã exit.

Dữ liệu và cấu hình đi đâu?

Container nên dễ tạo lại. Vì vậy dữ liệu cần giữ lâu thường đặt trong volume, còn cấu hình thay đổi theo môi trường nên truyền bằng env.



Host machine



Build



Run



Learn



VOLUME

Nơi Docker quản lý để lưu dữ liệu bền hơn vòng đời container.



BIND MOUNT

Map thư mục từ host vào container, hữu ích khi dev local.

Docker Compose chạy nhiều service

Khi app cần nhiều container như web, API và database, Compose giúp mô tả tất cả trong một file compose.yaml.

>_ COMPOSE.YAML

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
```

EXPECTED RESULT Một service web được mô tả bằng YAML

>_ COMMAND

```
docker compose up -d
```

EXPECTED RESULT Tạo network và chạy service nền

>_ CLEANUP

```
docker compose down
```

Bắt demo bằng Compose

>_ START SERVICES

```
cd Day2/example-app
docker compose up --build -d
```

EXPECTED RESULT API, Postgres, Redis, Prometheus và Grafana chạy nền

>_ CHECK SERVICES

```
cd Day2/example-app
docker compose ps
```

EXPECTED RESULT Các service chính ở trạng thái running hoặc healthy

>_ OPEN APP

```
http://localhost:3000
```

EXPECTED RESULT Orders App mở trên browser

Gọi API và tạo order

Sau khi các service chạy, mình dùng endpoint của example app để kiểm tra health, tạo dữ liệu mới và thấy Redis cache hoạt động qua stats.

> HEALTH

```
cd Day2/example-app
curl.exe http://localhost:3000/health
# Linux/macOS: curl http://localhost:3000/health
```

EXPECTED RESULT JSON status ok, database ok, redis ok

> CREATE ORDER

```
cd Day2/example-app
curl.exe -X POST http://localhost:3000/api/orders -H "Content-Type: application/json" -d '{"customer":"Demo team","item":"Docker lab","quantity":2}'
# Linux/macOS: dùng curl thay cho curl.exe
```

EXPECTED RESULT API trả về order mới với status created

> STATS

```
cd Day2/example-app
curl.exe http://localhost:3000/api/stats
# Linux/macOS: curl http://localhost:3000/api/stats
```


Xem logs, metrics và cleanup

Phần cuối demo dùng chính container đang chạy để xem logs API, mở metrics cho Prometheus và dọn các service Compose sau buổi học.

> LOGS

```
cd Day2/example-app
docker compose logs api --tail=40
```

EXPECTED RESULT Log startup và request API gần nhất

> METRICS

```
cd Day2/example-app
curl.exe http://localhost:3000/metrics
# Linux/macOS: curl http://localhost:3000/metrics
```

EXPECTED RESULT Prometheus metrics của API

> CLEANUP

```
cd Day2/example-app
```